

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA ĐÀO TẠO SAU ĐẠI HỌC



BÁO CÁO

CÁC HỆ THỐNG PHÂN TÁN

**Đề Tài: Phân tích và thiết kế hệ thống thương mại điện tử dựa trên kiến trúc
Microservices**

Giảng viên hướng dẫn	: TS. Kim Ngọc Bách
Lớp	: M25CQHT01-B
Nhóm thực hiện	: Nhóm 6
Đoàn Văn Nam	: B25CHHT042
Đông Quốc Huy	: B25CHHT028
Nguyễn Minh Tuấn	: B25CHHT062

Hà Nội – 2025

MỤC LỤC

I. Tổng quan	4
1. Bối cảnh sử dụng Microservices	4
2. Đặc điểm kỹ thuật nổi bật của hệ thống	4
3. Kiến trúc Microservices	4
4. Docker và Công nghệ Container hóa (Containerization)	5
4.1. Nguyên lý vận hành của Docker và Containerization	5
4.2. Docker Compose	6
4.3. Lợi ích thực tiễn đối với hệ thống E-commerce	6
5. Kafka và Mô hình Giao tiếp hướng sự kiện (Event-Driven)	7
5.1 Cơ chế vận hành chi tiết: Publish/Subscribe & Data Streaming	7
5.2 Vai trò chiến lược trong hệ thống phân tán	7
II. Phân tích yêu cầu hệ thống	8
1. Tổng quan hệ thống	8
2. Yêu cầu phi chức năng (Non-Functional Requirements)	8
2.1. Tính độc lập và Tự chủ (Autonomy)	8
2.2. Khả năng chịu lỗi (Fault Tolerance) & Sẵn sàng (Availability)	8
2.3. Hiệu năng và Khả năng mở rộng (Scalability)	8
3. Yêu cầu về Kiến trúc & Hạ tầng (Architectural & Infrastructure Constraints)	9
III. NHIỆM VỤ CỦA CÁC SERVICE	9
Xác định và mô tả các dịch vụ cụ thể	9
1. Dịch vụ Xác thực (Auth Service)	9
2. Dịch vụ Sản phẩm (Product Service)	9
3. Dịch vụ Đơn hàng (Order Service)	10
4. Dịch vụ Thanh toán (Payment Service)	10
5. API Gateway (Thành phần điều phối)	10
6. Message Broker (Kafka)	10
IV. Thiết kế phần mềm	11
1. Công cụ và công nghệ áp dụng	11
2. Thiết kế kiến trúc chung của hệ thống Microservice	11
2.1 Cấu trúc code	11
V. Quản lý	12
1. Cách quản lý từng dịch vụ	12
2. Mô hình triển khai tự động và liên tục (CI/CD)	12
3. Giải pháp theo dõi và khắc phục sự cố trong kiến trúc Microservices	13
VI. Kết quả và nhận xét	13
1. Kết quả đạt được	13
2. Những khó khăn và hạn chế	14

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa hiện nay, thương mại điện tử (E-commerce) không còn là một khái niệm xa lạ mà đã trở thành huyết mạch của nền kinh tế toàn cầu. Với sự bùng nổ về số lượng người dùng và khối lượng giao dịch khổng lồ, các hệ thống thương mại điện tử đòi hỏi sự linh hoạt, khả năng mở rộng cực cao và tính sẵn sàng liên tục.

Trước đây, các hệ thống thường được xây dựng dựa trên kiến trúc Monolithic (kiến trúc đơn khối). Tuy nhiên, khi quy mô hệ thống lớn dần, kiến trúc này bộc lộ nhiều hạn chế đáng kể như: khó khăn trong việc bảo trì, tốc độ triển khai chậm và rủi ro "một điểm lỗi gây hỏng toàn bộ hệ thống" (Single Point of Failure). Để giải quyết những thách thức này, kiến trúc Microservices (kiến trúc vi dịch vụ) đã nổi lên như một giải pháp tối ưu cho các hệ thống phân tán hiện đại.

Việc phân chia một hệ thống thương mại điện tử phức tạp thành các dịch vụ nhỏ độc lập—như dịch vụ quản lý giỏ hàng, dịch vụ thanh toán, dịch vụ kho bãi—không chỉ giúp các nhóm phát triển làm việc song song hiệu quả mà còn cho phép hệ thống tự động mở rộng từng phần theo nhu cầu thực tế. Tuy nhiên, việc chuyển dịch sang Microservices cũng đặt ra những bài toán hóc búa về quản lý dữ liệu phân tán, tính nhất quán (consistency) và sự phức tạp trong giao tiếp giữa các dịch vụ.

Bài báo cáo tập trung vào việc làm rõ các thành phần cốt lõi của hệ thống, cách thức thiết kế các dịch vụ độc lập và các chiến lược đảm bảo hệ thống vận hành ổn định trong môi trường phân tán. Mặc dù đã có nhiều cố gắng trong quá trình tìm hiểu và thực hiện, song do kiến thức và kinh nghiệm còn hạn chế, bài báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự đóng góp ý kiến từ thầy để đề tài được hoàn thiện hơn.

I. Tổng quan

1. Bối cảnh sử dụng Microservices

Kiến trúc Microservices được áp dụng trong dự án nhằm khắc phục các nhược điểm trên. Hệ thống được chia thành các dịch vụ độc lập như user-service, product-service, order-service, mỗi dịch vụ có vòng đời phát triển và triển khai riêng, giúp tăng tính linh hoạt và ổn định.

2. Đặc điểm kỹ thuật nổi bật của hệ thống

Dựa trên cấu trúc dự án, kiến trúc Microservices mang lại các lợi ích chính sau:

- **Tính độc lập và chịu lỗi:** Mỗi service hoạt động độc lập, lỗi của một dịch vụ không làm gián đoạn toàn hệ thống.
- **Khả năng mở rộng linh hoạt:** Có thể mở rộng riêng từng service khi xảy ra tải cao (ví dụ inventory-service trong đợt khuyến mãi).
- **Đa dạng công nghệ:** Mỗi dịch vụ có thể sử dụng công nghệ và cơ sở dữ liệu phù hợp với nghiệp vụ của mình.

3. Kiến trúc Microservices

Microservices là kiến trúc trong đó một ứng dụng lớn được chia thành nhiều dịch vụ nhỏ, độc lập, giao tiếp với nhau thông qua các giao thức nhẹ như HTTP hoặc Message Queue. Mỗi dịch vụ đảm nhiệm một chức năng nghiệp vụ riêng và có thể phát triển, triển khai độc lập.

Kiến trúc này mang lại các đặc trưng nổi bật như: liên kết lỏng lẻo, triển khai độc lập, khả năng mở rộng cao, cô lập lỗi, quản lý dữ liệu phi tập trung và khả năng chịu lỗi tốt.

Các đặc trưng của kiến trúc Microservice:

- Phân tán dữ liệu (Decentralized Data):

Thay vì sử dụng một cơ sở dữ liệu khổng lồ dùng chung (như kiến trúc Monolithic), mỗi microservice quản lý cơ sở dữ liệu riêng của nó. Điều này giúp tránh hiện tượng "điểm nghẽn" và tăng tính độc lập.

- Tính tự trị (Autonomy):

Mỗi dịch vụ có thể được phát triển, triển khai và mở rộng quy mô một cách độc lập mà không cần chờ đợi hay ảnh hưởng đến các phần khác của hệ thống.

- Tính lỏng lẻo (Loosely Coupled)

Microservices giao tiếp với nhau thông qua các giao thức nhẹ như REST (Representational State Transfer) hoặc hàng đợi thông điệp. Các dịch vụ được tách rời nhau, cho phép phát triển, triển khai và mở rộng một cách độc lập.

- Phát triển độc lập (Independent Deployment)

Microservices có thể được triển khai độc lập với nhau. Điều này có nghĩa là thay đổi hoặc cập nhật một dịch vụ không yêu cầu phải triển khai lại toàn bộ ứng dụng, giảm thiểu thời gian ngừng hoạt động và sự gián đoạn.

“Product not project” đây là khái niệm của Amazon “bạn xây dựng, bạn vận hành”, nơi một nhóm phát triển chịu trách nhiệm toàn bộ cho phần mềm trong quá trình hoạt động.

- Khả năng mở rộng (Scalability)

Có thể chỉ mở rộng những dịch vụ đang bị quá tải thay vì phải nhân bản toàn bộ ứng dụng.

- Khả năng cô lập lỗi (Fault Isolation)

Nếu một dịch vụ gặp sự cố, nó không ảnh hưởng đến toàn bộ ứng dụng. Sự cố được giới hạn trong dịch vụ gặp lỗi, và các dịch vụ khác vẫn có thể hoạt động bình thường.

- Đa dạng công nghệ (Technology Diversity)

Các dịch vụ khác nhau có thể được viết bằng các ngôn ngữ lập trình khác nhau hoặc sử dụng các công nghệ khác nhau. Điều này giúp dễ dàng chọn công cụ phù hợp cho từng công việc, thay vì bị ràng buộc vào một nền tảng công nghệ cụ thể.

- Thiết kế để chịu lỗi và khả năng hồi phục (Resilience)

Microservices được thiết kế để đối phó với các lỗi và quản lý lỗi với các hành động phù hợp. Microservices cũng được thiết kế để phục hồi, nghĩa là chúng có thể tiếp tục hoạt động ngay cả khi một hoặc nhiều dịch vụ bị lỗi. Vì mỗi dịch vụ hoạt động độc lập, sự cố trong một dịch vụ không ảnh hưởng đến toàn bộ ứng dụng.

- Quản lý dữ liệu phi tập trung (Decentralized Data Management)

Microservices cũng phân cấp quyết định lưu trữ dữ liệu. Chúng ta có thể gọi cách tiếp cận này là Polyglot Persistence hoặc Polyglot Databases. Điều này có nghĩa là Microservices cho phép mỗi dịch vụ quản lý cơ sở dữ liệu của riêng mình, có thể là các phiên bản khác nhau của cùng một công nghệ cơ sở dữ liệu hoặc các hệ thống cơ sở dữ liệu hoàn toàn khác nhau.

- Tăng tốc độ triển khai:

Phù hợp với các mô hình DevOps và CI/CD, cho phép phát hành các tính năng mới nhanh chóng

4. Docker và Công nghệ Container hóa (Containerization)

Trong bối cảnh hệ thống Microservices bao gồm nhiều dịch vụ riêng lẻ (order-service, payment-service, inventory-service,...), việc triển khai thủ công từng dịch vụ là bất khả thi. Docker đóng vai trò là "tiêu chuẩn hóa" đơn vị phần mềm, giúp đóng gói ứng dụng và tất cả các thành phần phụ thuộc vào một môi trường cô lập.

4.1. Nguyên lý vận hành của Docker và Containerization

Docker hoạt động dựa trên cơ chế chia sẻ nhân hệ điều hành (Host OS Kernel) nhưng vẫn đảm bảo sự cô lập hoàn toàn giữa các tiến trình.

- **Tính đóng gói (Encapsulation):** Mỗi Microservice (ví dụ: api-gateway) được định nghĩa trong một Dockerfile. File này chứa các chỉ dẫn để xây dựng Docker Image – một snapshot tĩnh chứa từ mã nguồn các biến môi trường cho đến các thư viện hệ thống cần thiết.
- **Tính bất biến (Immutability):** Một khi Image đã được tạo ra, nó sẽ không thay đổi. Điều này loại bỏ hoàn toàn lỗi "it works on my machine" (chạy tốt trên máy tôi nhưng lỗi trên server) do sai lệch phiên bản thư viện hoặc môi trường.
- **Tiết kiệm tài nguyên:** Vì không cần khởi chạy một hệ điều hành khách (Guest OS) như máy ảo (VM), các Docker Container khởi động chỉ trong vài giây và tiêu tốn rất ít RAM/CPU, cho phép chạy hàng chục service trên cùng một máy chủ vật lý.

4.2. Docker Compose

Đóng vai trò điều phối toàn bộ hệ thống, quản lý sự phụ thuộc giữa các dịch vụ, thiết lập mạng nội bộ, cấu hình biến môi trường và lưu trữ dữ liệu bền vững.

Các chức năng cốt lõi của Docker Compose trong dự án:

- **Quản lý sự phụ thuộc (Dependency Management):** Thông qua chỉ thị `depends_on`, Docker Compose đảm bảo các dịch vụ hạ tầng như Kafka phải khởi động thành công trước khi các business service như product-service bắt đầu chạy. Điều này ngăn ngừa lỗi kết nối khi hệ thống vừa khởi động.
- **Hệ thống mạng nội bộ (Internal Networking):** Docker Compose tự động khởi tạo một mạng ảo (Virtual Bridge Network). Trong mạng này, các container có thể giao tiếp với nhau bằng **Service Name** thay vì địa chỉ IP.
- **Quản lý biến môi trường tập trung:** Thay vì cấu hình cứng (hard-code) các tham số như thông tin kết nối Database hay địa chỉ Kafka Broker vào mã nguồn, Docker Compose cho phép định nghĩa chúng trong phần `environment`. Điều này giúp hệ thống linh hoạt khi chuyển đổi giữa môi trường Development, Staging và Production.
- **Quản lý dữ liệu bền vững (Volumes):** Vì Container có đặc tính "tạm thời" (dữ liệu biến mất khi container bị xóa), Docker Compose sử dụng cơ chế volumes để gắn kết (map) dữ liệu từ bên trong container (như dữ liệu MySQL của user-service) ra đĩa cứng của máy chủ. Điều này đảm bảo dữ liệu đơn hàng và người dùng luôn được bảo toàn ngay cả khi nâng cấp hoặc khởi động lại hệ thống.

4.3. Lợi ích thực tiễn đối với hệ thống E-commerce

1. **Mở rộng nhanh chóng (Scaling):** Với Docker, ta có thể dễ dàng tăng số lượng instance cho search-service chỉ bằng lệnh `docker-compose up --scale search-service=3` để đáp ứng lượng truy cập lớn mà không cần cấu hình lại mạng.
2. **Khôi phục thảm họa (Disaster Recovery):** Nếu một container bị sập, Docker có thể được cấu hình để tự động khởi động lại (restart policy), giúp hệ thống tự phục hồi mà không cần can thiệp thủ công.
3. **Môi trường phát triển đồng nhất:** Thành viên mới gia nhập dự án chỉ cần cài đặt Docker và chạy một lệnh duy nhất để có toàn bộ môi trường hoạt động (gồm DB, Message Broker, các Service), giảm thời gian thiết lập từ vài ngày xuống vài phút.

5. Kafka và Mô hình Giao tiếp hướng sự kiện (Event-Driven)

Trong một hệ thống phân tán phức tạp như Thương mại điện tử, việc các dịch vụ gọi trực tiếp lẫn nhau (Point-to-point) sẽ tạo ra một mạng lưới kết nối chằng chịt và dễ đổ vỡ. Apache Kafka xuất hiện như một hệ thống trung tâm giúp tách rời các dịch vụ (Decoupling).

5.1 Cơ chế vận hành chi tiết: Publish/Subscribe & Data Streaming

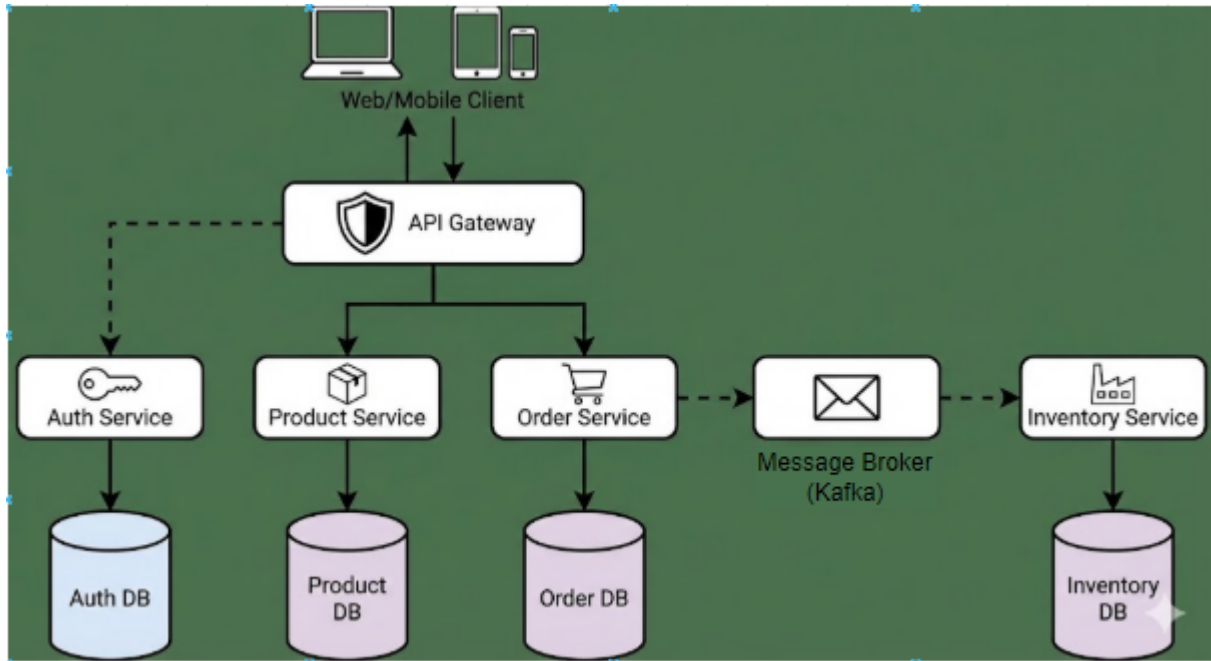
Apache Kafka được sử dụng để hỗ trợ giao tiếp bất đồng bộ giữa các dịch vụ, giúp giảm sự phụ thuộc trực tiếp và tăng khả năng mở rộng hệ thống. Thông qua mô hình **Publish/Subscribe**, các dịch vụ có thể gửi và nhận sự kiện mà không cần biết trực tiếp nhau.

Kafka giúp hệ thống xử lý song song hiệu quả, đảm bảo khả năng chịu tải cao, phục hồi dữ liệu và nâng cao trải nghiệm người dùng trong các nghiệp vụ phức tạp như xử lý đơn hàng.

5.2 Vai trò chiến lược trong hệ thống phân tán

- **Xử lý bất đồng bộ (Asynchronous Processing) & Nâng cao trải nghiệm người dùng:**
 - Trong luồng đặt hàng, nếu thực hiện đồng bộ, người dùng phải đợi: Lưu đơn -> Trừ kho -> Tính thuế -> Gửi Email -> Phản hồi. Chỉ cần một bước chậm, người dùng sẽ thấy ứng dụng bị "treo".
 - Với Kafka, order-service chỉ làm nhiệm vụ chính là lưu đơn và bắn một Event vào Kafka, sau đó phản hồi ngay cho người dùng: "Đơn hàng của bạn đang được xử lý". Các tác vụ nặng như gửi Email hay tính toán vận chuyển sẽ được thực hiện "ngầm" bởi các dịch vụ khác ở phía sau.
- **Khả năng chịu tải và Mở rộng (Scalability qua Partitions):**
 - Kafka chia một Topic thành nhiều **Partitions** (Phân vùng). Các phân vùng này được phân tán trên nhiều máy chủ (Brokers) khác nhau.
 - Cơ chế này cho phép hệ thống thực hiện song song hóa: 10 instance của **notification-service** có thể đọc đồng thời từ 10 partitions khác nhau của cùng một Topic, giúp tốc độ xử lý tăng gấp 10 lần mà không bị thắt nút cổ chai.
- **Khả năng chịu lỗi (Fault Tolerance):**
 - Kafka tự động sao lưu (Replication) mỗi Partition sang nhiều Broker khác nhau. Nếu một máy chủ chứa Kafka bị sập, hệ thống vẫn hoạt động bình thường vì dữ liệu đã có bản sao ở máy chủ khác.

II. Phân tích yêu cầu hệ thống



1. Tổng quan hệ thống

Hệ thống được thiết kế theo kiến trúc Microservices, nhằm giải quyết các hạn chế của mô hình Monolithic truyền thống như khả năng mở rộng kém và sự phụ thuộc chặt chẽ giữa các thành phần. Mục tiêu cốt lõi là xây dựng một nền tảng thương mại điện tử phi tập trung, trong đó các dịch vụ nghiệp vụ (Services) hoạt động độc lập, giao tiếp lỏng lẻo (Loosely Coupled) và có khả năng chịu lỗi cao.

2. Yêu cầu phi chức năng (Non-Functional Requirements)

2.1. Tính độc lập và Tự chủ (Autonomy)

- Database per Service: Mỗi dịch vụ phải sở hữu một cơ sở dữ liệu riêng biệt. Tuyệt đối không chia sẻ chung Database schema. Việc truy xuất dữ liệu chéo phải thực hiện thông qua API hoặc cơ chế nhân bản dữ liệu (Data Replication).
- Technology Agnostic: Các dịch vụ có thể được phát triển trên các ngôn ngữ lập trình và nền tảng công nghệ khác nhau (NodeJS, Python, Java...) tùy thuộc vào đặc thù xử lý.

2.2. Khả năng chịu lỗi (Fault Tolerance) & Sẵn sàng (Availability)

- Hệ thống phải đảm bảo Isolation (Cô lập lỗi): Nếu *Inventory Service* gặp sự cố ngừng hoạt động, *Order Service* vẫn phải tiếp nhận được đơn hàng (lưu vào hàng đợi) và không được trả lỗi cho người dùng.
- Sử dụng cơ chế Retry và Circuit Breaker khi gọi các dịch vụ nội bộ.

2.3. Hiệu năng và Khả năng mở rộng (Scalability)

- Hệ thống hỗ trợ Horizontal Scaling (Mở rộng theo chiều ngang). Ví dụ: Có thể chạy 5 container cho *Order Service* vào dịp Sale, trong khi chỉ cần 1 container cho *Product Service*, nhờ vào việc container hóa (Docker).
- Giảm độ trễ phản hồi (Latency) bằng cách chuyển đổi các tác vụ nặng sang xử lý bất đồng bộ (Asynchronous processing) qua RabbitMQ/Kafka.

3. Yêu cầu về Kiến trúc & Hạ tầng (Architectural & Infrastructure Constraints)

- Mô hình triển khai: Ứng dụng được đóng gói (Containerization) sử dụng Docker và điều phối bằng Docker Compose (cho môi trường Dev/Test).
- Cơ chế giao tiếp:
 - Giao tiếp Frontend - Backend: RESTful API (JSON).
 - Giao tiếp Backend - Backend: Kết hợp giữa HTTP (Sync) cho các tác vụ cần phản hồi ngay và AMQP (Async) cho các tác vụ nền.
- Quy trình CI/CD:
 - Tự động hóa quy trình tích hợp và triển khai.
 - Môi trường Build: Linux (tối ưu cho Docker).
 - Môi trường Production: Windows Server (tận dụng hạ tầng có sẵn) thông qua cơ chế Reverse Proxy (IIS) tới máy ảo Linux.

III. NHIỆM VỤ CỦA CÁC SERVICE

Xác định và mô tả các dịch vụ cụ thể

Hệ thống được xây dựng theo kiến trúc microservices, trong đó mỗi dịch vụ đảm nhiệm một chức năng nghiệp vụ riêng biệt. Việc tách các chức năng thành các dịch vụ độc lập giúp hệ thống dễ mở rộng, dễ bảo trì và tăng tính linh hoạt trong quá trình phát triển.

1. Dịch vụ Xác thực (Auth Service)

- Nhiệm vụ: Quản lý toàn bộ vòng đời của người dùng và bảo mật hệ thống.
- Chức năng chính:
 - Đăng ký tài khoản mới và lưu trữ thông tin vào User DB.
 - Xác thực đăng nhập và cấp phát mã định danh (thường là JWT - JSON Web Token).
 - Quản lý quyền hạn (Phân quyền Admin/User).
 - Đảm bảo các yêu cầu từ Client qua API Gateway là hợp lệ.

2. Dịch vụ Sản phẩm (Product Service)

- Nhiệm vụ: Quản lý danh mục hàng hóa và hiển thị thông tin cho khách hàng.
- Chức năng chính:
 - Lưu trữ thông tin chi tiết về sản phẩm (tên, mô tả, hình ảnh, thông số) trong Catalog DB.
 - Quản lý danh mục sản phẩm và tìm kiếm.

- Cập nhật trạng thái sản phẩm (còn hàng/hết hàng).
- Cung cấp dữ liệu cho Client khi người dùng lướt xem trang web hoặc ứng dụng.

3. Dịch vụ Đơn hàng (Order Service)

- Nhiệm vụ: Điều phối quy trình mua hàng từ khi khách chọn mua đến khi đơn hàng hoàn tất.
- Chức năng chính:
 - Tạo và lưu trữ các đơn hàng mới vào Order DB.
 - Tính toán tổng giá trị đơn hàng, phí vận chuyển.
 - Quản lý trạng thái đơn hàng (Chờ thanh toán, Đã thanh toán, Đang giao, Đã hủy).
 - Giao tiếp: Đẩy thông điệp (Event) vào Message Broker để thông báo cho dịch vụ Thanh toán và dịch vụ Kho.

4. Dịch vụ Thanh toán (Payment Service)

- Nhiệm vụ: Xử lý các giao dịch tài chính một cách an toàn và tin cậy.
- Chức năng chính:
 - Kết nối với các cổng thanh toán bên thứ ba (VNPay, Momo, Visa/Mastercard).
 - Ghi lại lịch sử giao dịch vào Payment DB.
 - Xử lý hoàn tiền khi đơn hàng bị hủy.
 - Lắng nghe tín hiệu từ Message Broker để kích hoạt thanh toán ngay khi đơn hàng được tạo.

5. API Gateway (Thành phần điều phối)

Mặc dù không chứa nghiệp vụ bán hàng, nhưng nó có nhiệm vụ "cửa ngõ" quan trọng:

- Nhiệm vụ: Tiếp nhận mọi yêu cầu từ Client và điều hướng (Routing) đúng đến dịch vụ tương ứng.
- Chức năng chính: Cân bằng tải (Load Balancing), giới hạn tốc độ truy cập (Rate Limiting) và ngăn chặn các cuộc tấn công cơ bản.

6. Message Broker (Kafka)

- Nhiệm vụ: Đảm bảo tính nhất quán sau cùng (Eventual Consistency) giữa các dịch vụ.
- Chức năng chính: Giúp các dịch vụ giao tiếp bất đồng bộ. Ví dụ: Khi Order Service xong nhiệm vụ, nó gửi tin nhắn vào đây, Payment Service sẽ tự động "nhặt" tin nhắn đó để xử lý mà không cần Order Service phải chờ đợi, giúp hệ thống không bị tắc nghẽn.

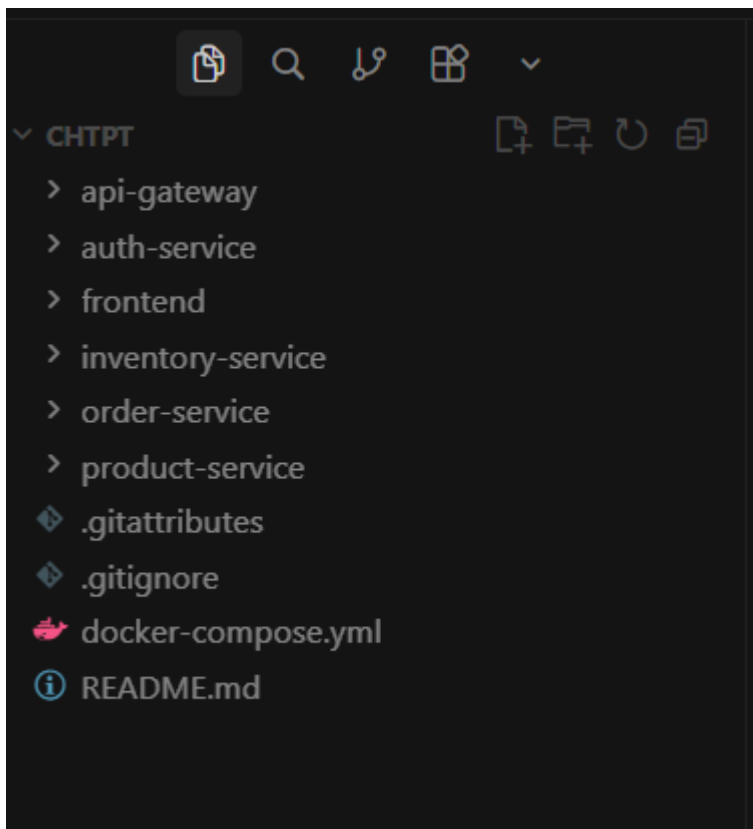
IV. Thiết kế phần mềm

1. Công cụ và công nghệ áp dụng

- Ngôn ngữ lập trình: dotNET, NodeJs
- Nền tảng chạy ứng dụng: dotnet SDK, nodejs
- Framework: NET 9.0, nodejs express
- Kiến trúc hệ thống: Microservices
- IDE / Trình soạn thảo mã nguồn: Visual Studio Code
- Công cụ kiểm thử API: Postman
- Quản lý mã nguồn: Git
- Lưu trữ mã nguồn: GitHub
- Công nghệ đóng gói & triển khai: Docker, Docker Compose
- Message Broker: Apache Kafka
- API Gateway: dotnet ocelot
- Cơ sở dữ liệu: MongoDB (dữ liệu phi cấu trúc, thông báo)
- Thiết kế hệ thống & sơ đồ: StarUML, Whimsical

2. Thiết kế kiến trúc chung của hệ thống Microservice

2.1 Cấu trúc code



Các service

- api-gateway: Điểm truy cập duy nhất, định tuyến request và xác thực JWT.
- auth-service: Xác thực và phân quyền.
- frontend: Ứng dụng web dành cho người dùng cuối.

- product-service: Quản lý và tra cứu thông tin sản phẩm.
- order-service: Tạo và quản lý đơn hàng, điều phối nghiệp vụ.
- inventory-service: Quản lý tồn kho, giữ và hoàn hàng.

File cấu hình & quản lý

- docker-compose.yml: Triển khai toàn bộ hệ thống.
- README.md: Mô tả và hướng dẫn dự án.

V. Quản lý

Trong kiến trúc Microservices, công tác quản lý hệ thống đóng vai trò then chốt nhằm đảm bảo hệ thống vận hành ổn định, dễ mở rộng và dễ bảo trì. Do hệ thống được chia thành nhiều dịch vụ độc lập, việc quản lý không chỉ dừng lại ở mức ứng dụng mà còn bao gồm quản lý vòng đời dịch vụ, triển khai, giám sát và xử lý sự cố.

1. Cách quản lý từng dịch vụ

Mỗi Microservice trong hệ thống được thiết kế và quản lý như một đơn vị độc lập, có vòng đời phát triển riêng biệt. Các nguyên tắc quản lý chính bao gồm:

- Quản lý độc lập theo service:
Mỗi dịch vụ (user-service, product-service, order-service,...) có mã nguồn, cấu hình và cơ sở dữ liệu riêng. Điều này giúp việc phát triển, sửa lỗi hoặc nâng cấp một dịch vụ không ảnh hưởng đến các dịch vụ còn lại.
- Quản lý truy cập tập trung thông qua API Gateway:
API Gateway đóng vai trò là điểm vào duy nhất của hệ thống, chịu trách nhiệm định tuyến request, kiểm tra xác thực và phân quyền. Cách tiếp cận này giúp đơn giản hóa việc quản lý bảo mật và kiểm soát truy cập cho toàn bộ hệ thống.
- Quản lý cấu hình linh hoạt:
Các thông tin cấu hình như cổng dịch vụ, thông tin kết nối cơ sở dữ liệu, Kafka broker được tách khỏi mã nguồn và cấu hình thông qua biến môi trường hoặc file cấu hình. Điều này giúp hệ thống dễ dàng triển khai trên nhiều môi trường khác nhau.

2. Mô hình triển khai tự động và liên tục (CI/CD)

Để đảm bảo hệ thống có thể phát triển và triển khai nhanh chóng, mô hình CI/CD được áp dụng nhằm tự động hóa quy trình build, test và triển khai.

- Continuous Integration (CI):
Mỗi khi có thay đổi mã nguồn, hệ thống CI sẽ tự động:

- Kiểm tra mã nguồn

- o Build từng service độc lập
- o Chạy các bài kiểm tra cơ bản (unit test, integration test)
- Continuous Deployment (CD):
Sau khi build thành công, các service có thể được đóng gói dưới dạng Docker Image và triển khai lên môi trường chạy. Việc sử dụng Docker giúp đảm bảo tính nhất quán giữa môi trường phát triển và môi trường triển khai.
- Triển khai độc lập từng service:
Nhờ kiến trúc Microservices, việc cập nhật một dịch vụ (ví dụ product-service) không yêu cầu dừng toàn bộ hệ thống. Điều này giúp giảm thời gian gián đoạn và nâng cao tính sẵn sàng của hệ thống.
- Tự động hóa triển khai:
Docker Compose (hoặc các công cụ điều phối khác như Kubernetes trong thực tế) giúp tự động hóa quá trình khởi động và liên kết giữa các service, giảm thiểu sai sót do cấu hình thủ công.

3. Giải pháp theo dõi và khắc phục sự cố trong kiến trúc Microservices

Do hệ thống Microservices có nhiều thành phần phân tán, việc theo dõi và xử lý sự cố cần được thiết kế ngay từ đầu.

- Theo dõi trạng thái dịch vụ (Health Check):
Mỗi service cung cấp endpoint kiểm tra trạng thái (health check). Điều này cho phép phát hiện sớm các dịch vụ gặp sự cố và thực hiện các hành động khôi phục.
- Giám sát và logging tập trung:
Log của các service được ghi lại nhằm phục vụ việc phân tích lỗi và theo dõi hành vi hệ thống. Trong hệ thống thực tế, các công cụ như ELK Stack (Elasticsearch, Logstash, Kibana), Seq có thể được sử dụng để tập trung hóa và trực quan hóa log.
- Cơ chế chịu lỗi và phục hồi:
Các service được thiết kế theo hướng chịu lỗi, trong đó nếu một service gặp sự cố, các service khác vẫn có thể tiếp tục hoạt động. Docker hỗ trợ tự động khởi động lại container khi gặp lỗi, giúp hệ thống nhanh chóng phục hồi.

VI. Kết quả và nhận xét

1. Kết quả đạt được

- Xây dựng thành công mô hình kiến trúc Microservices cho hệ thống thương mại điện tử:
Hệ thống được chia thành nhiều dịch vụ độc lập như quản lý người dùng, sản phẩm, đơn hàng, thanh toán,... mỗi dịch vụ đảm nhiệm một chức năng riêng biệt, đúng với nguyên tắc của kiến trúc Microservices.
- Áp dụng giao tiếp bất đồng bộ thông qua hệ thống message broker:
Việc sử dụng Kafka giúp các dịch vụ giao tiếp với nhau theo mô hình bất đồng

bộ, giảm sự phụ thuộc trực tiếp giữa các service, từ đó nâng cao khả năng mở rộng và độ ổn định của hệ thống.

- Đóng gói và triển khai hệ thống bằng Docker:
Các thành phần hạ tầng như cơ sở dữ liệu và message broker được triển khai bằng Docker, giúp quá trình cài đặt và chạy hệ thống trở nên nhất quán, dễ dàng và giảm phụ thuộc vào môi trường.
- Đáp ứng được các đặc trưng của hệ thống phân tán:
Hệ thống thể hiện rõ các đặc điểm của hệ thống phân tán như phân tán dữ liệu, phân tán xử lý, khả năng mở rộng và khả năng chịu lỗi ở mức cơ bản.

2. Những khó khăn và hạn chế

- Độ phức tạp trong quản lý và cấu hình hệ thống:
Do hệ thống bao gồm nhiều service độc lập, việc cấu hình và đồng bộ các thành phần đòi hỏi nhiều công sức hơn so với kiến trúc đơn khối (Monolithic).
- Khó khăn trong quá trình tích hợp và debug:
Khi xảy ra lỗi trong một nghiệp vụ phức tạp liên quan đến nhiều service, việc xác định nguyên nhân gốc gặp nhiều khó khăn nếu thiếu các công cụ giám sát và theo dõi chuyên sâu.
- Hệ thống chưa được triển khai đầy đủ các cơ chế nâng cao:
Các giải pháp như cân bằng tải tự động, giám sát hiệu năng nâng cao, bảo mật phân tán hoặc triển khai trên nền tảng Kubernetes mới chỉ dừng lại ở mức định hướng, chưa được triển khai toàn diện trong phạm vi bài tập.
- Giới hạn về quy mô thử nghiệm:
Hệ thống chủ yếu được triển khai và kiểm thử trong môi trường học tập, với số lượng người dùng và dữ liệu còn hạn chế, chưa phản ánh đầy đủ các thách thức của hệ thống phân tán ở quy mô lớn.